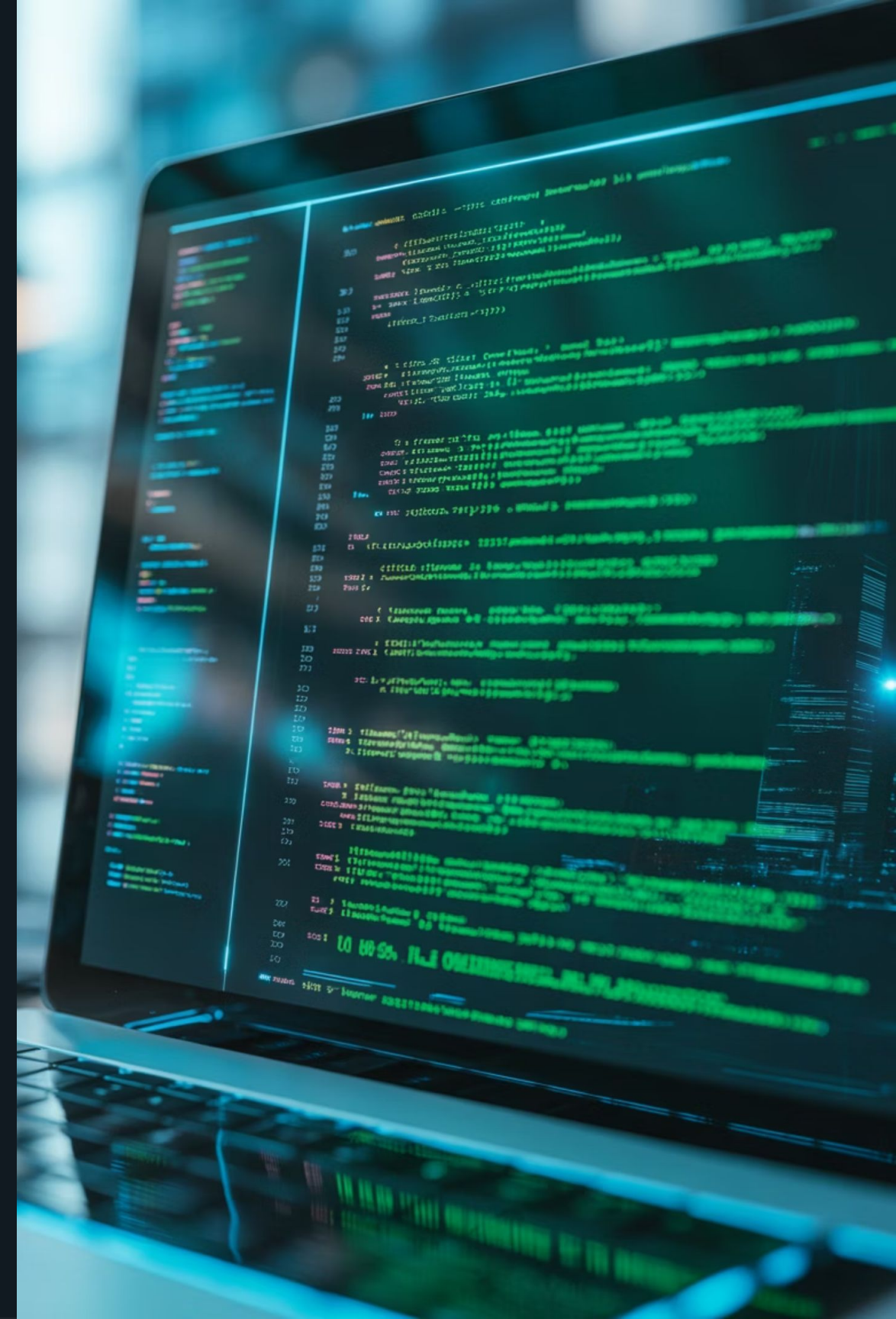


JavaScript Assíncrono, Express e Prisma



Introdução ao Assíncronismo no JavaScript

Código Síncrono vs. Assíncrono

Síncrono: Executa uma tarefa por vez, na ordem sequencial. O programa "espera" cada operação terminar antes de seguir para a próxima.

Assíncrono: Permite que operações sejam iniciadas e continuem em segundo plano enquanto o programa segue executando outras tarefas.

```
// Código síncrono
console.log("Primeiro");
console.log("Segundo");
// Código assíncrono
console.log("Primeiro");
setTimeout(() => { console.log("Terceiro (depois de 2s)"); }, 2000);
console.log("Segundo");
```

Event Loop e Call Stack

O **Event Loop** é o mecanismo que permite JavaScript executar operações não-bloqueantes, mesmo sendo single-threaded.



Promises

Uma **Promise** é um objeto que representa a eventual conclusão (ou falha) de uma operação assíncrona e seu valor resultante.

Estados de uma Promise

Pending: Estado inicial, operação em andamento

Fulfilled: Operação concluída com sucesso

Rejected: Operação falhou

Métodos Principais

.then(): Executa quando a promise é resolvida

.catch(): Captura erros na promise

.finally(): Executa independente do resultado

```
// Exemplo de Promise simulando tempo de espera
function esperarTempo(ms) { return new Promise((resolve, reject) => { setTimeout(() => { resolve(`Esperou ${ms}ms`);
// Para simular erro:
reject(new Error('Falhou')); }, ms); }));}
esperarTempo(2000).then(resultado => console.log(resultado)).catch(erro => console.error(erro)).finally(() => console.log('Finalizado'));
```

Async e Await

Promise Tradicional

```
function buscarDados() { return fetch('api/dados')  
.then(resposta => resposta.json()) .then(dados => {  
console.log(dados); return dados; }) .catch(erro => {  
console.error(erro); });}
```

Com Async/Await

```
async function buscarDados() { try { const resposta = await  
fetch('api/dados'); const dados = await  
resposta.json(); console.log(dados); return dados; }  
catch (erro) { console.error(erro); }}
```

Vantagens do Async/Await

- Código mais limpo e legível
- Estrutura similar ao código síncrono
- Melhor tratamento de erros com try/catch

Pontos Importantes

- Uma função **async** sempre retorna uma Promise
- await** só pode ser usado dentro de funções async
- Erros são propagados naturalmente